

COMP90077
Lecture 12
Online Algorithms + Wrap Up

William Umboh
University of Melbourne

Caution

- Slides are designed as presentation aid to go along with lecture, not as study material
- Slides not designed to make sense on their own

Assignment 2 (Deadline Time Change)

- Due: Saturday 30 May ~~17:00~~ **23:59**
- Problems conceptually harder than A1
- Take a look ASAP if you have not already
- Problems usually require subconscious processing
- Spend half an hour to load the problems so that your subconscious can work on it in the background
- Any partners?
- Feel free to approach me over email/consultation if you are not sure of level of details required in responses (proofs, reflection, etc)

FETT Short. Extension
Request Form
Automatic 3-calendar
day extension.

Today

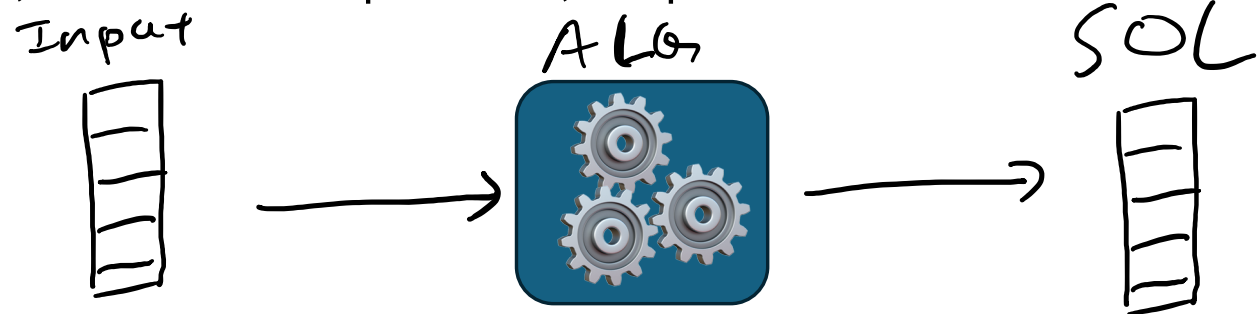
- Last lecture



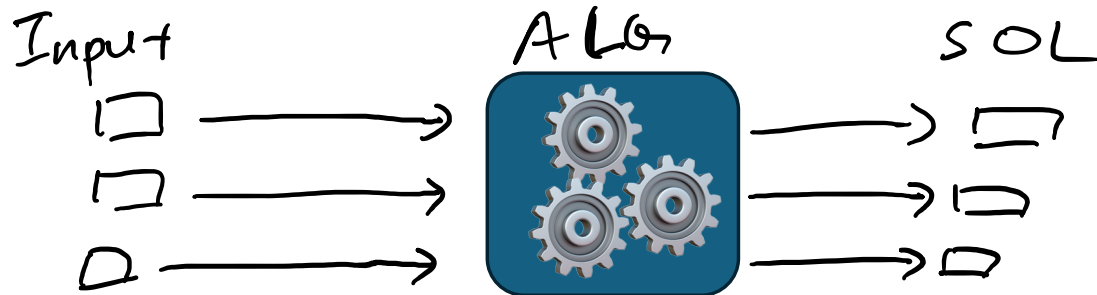
- Online Algorithms
 - Randomized Lower Bounds for Tram Waiting and Set Cover
 - Randomized Algorithm for Tram Waiting Problem via LP
- Recap
- Themes and Takeaways
- Exam
- AMA?

Recap: Online Algorithms

- Traditional computational model:
 - Given entire input, do some computation, output solution



- Online algorithms model:
 - In each timestep, some input arrives and algorithm takes some action

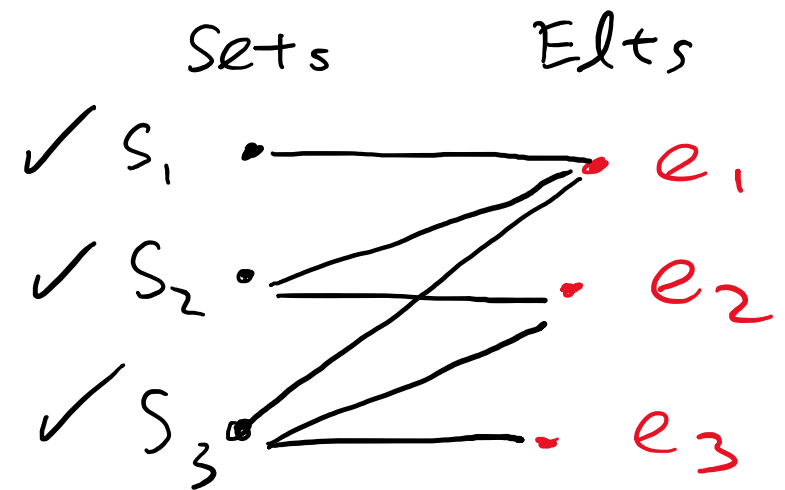
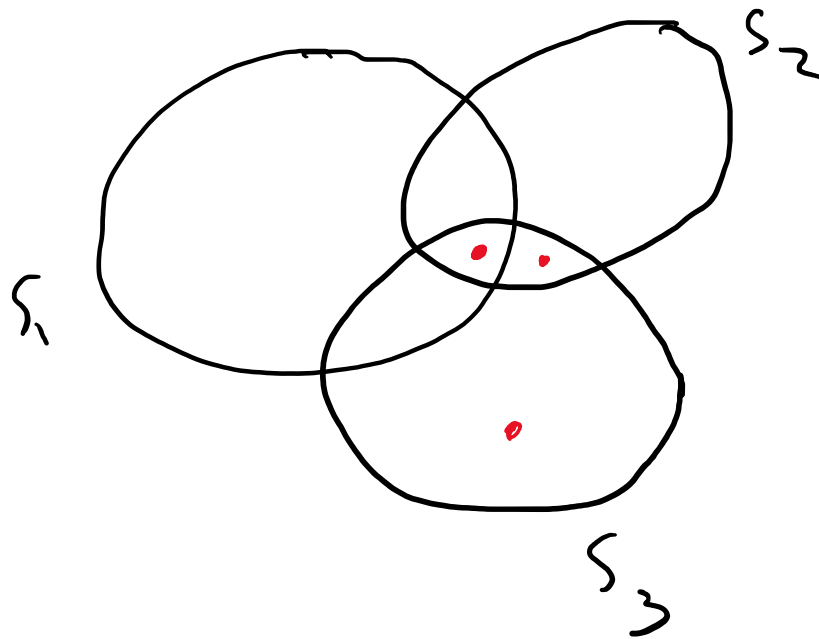


Recap: Tram Waiting

- Note: Actual problem name is Rent or Buy (aka Ski Rental)

Tram?	Alg	Cost	
No	wait	1	ALG = 2 + W OPT = min{3W}
No	wait	1	
No	walk	W	
Yes	ii		

Recap: Set Cover



ALG = 3
OPT = 1

Recap: Competitive Analysis

- Worst-Case Analysis for Online Algorithms
- **Competitive ratio**: ALG/OPT where OPT is the optimal solution with full information (aka offline optimal)
- **Minimization**: Algorithm is α -competitive if $cost \leq \alpha OPT$ on every instance
- **Maximization**: Algorithm is α -competitive if $value \geq \alpha OPT$ on every instance
- **Observe:**
 - No historical data
 - No machine learning
 - Future is adversarial

Approximation algorithms: Time vs Solution Quality
Online algorithms: Information vs Solution Quality

Deterministic Lower Bounds



- Tram Waiting: 2
- Set Cover: m = number of sets
- Fractional Set Cover: $\Omega(\log m)$

Randomized Algorithms



- Online Set Cover:

- $O(\log m \log n)$ in polynomial time via fractional set cover
- $o(\log m \log n)$ in polynomial time is NP-hard
- Can we do better with unbounded computational resources?

Randomized vs Fractional Algorithms

- Deterministic fractional at least as powerful as randomized integral
- Fractional algorithm can simulate randomized algorithm
- For Set Cover:
 - For each set S_i , let p_i = probability that randomized alg. has already bought S_i
 - Fractional variable $x_i = p_i$
- General Idea:
 - Randomized algorithm = probability distribution over solutions
 - Fractional variables = probabilities
- Randomized comp ratio $\geq$ Fractional comp ratio
- Online Set Cover: Randomized comp ratio $\geq \Omega(\log m)$

Randomized Algorithm for Tram Waiting

Thm There is a randomized $(\frac{e}{e-1})$ -competitive alg
for Train Waiting Problem.

Step 1 Solve fractional version

Step 2 Rounding.

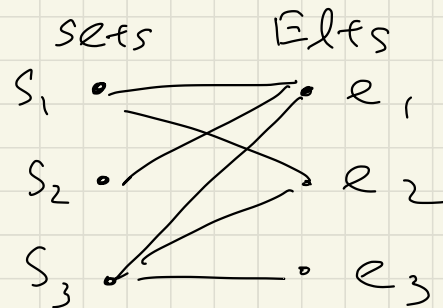
Fractional version

$$x_i = \begin{cases} 1 & \text{if walk at timestep } i \\ 0 & \end{cases}$$

$$\min \sum_i (W + i-1) x_i \quad \text{Valid LP relaxation.}$$

$$\text{s.t.} \quad \sum_i x_i \geq 1$$

$$\begin{aligned}
 \min \quad & x_1 + x_2 + x_3 \\
 \text{s.t.} \quad & x_1 + x_2 + x_3 \geq 1 \\
 & x_1 + x_3 \geq 1 \\
 & x_3 \geq 1
 \end{aligned}$$

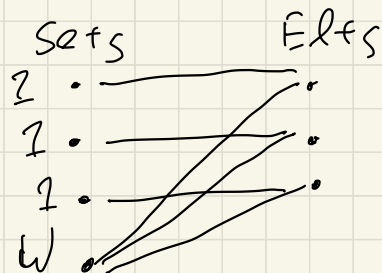


Tram Waiting as Set Cover

TimeSteps $\longleftrightarrow$ Elements

wait at timestep i $\longleftrightarrow$ set $\{i\}$ cost 1

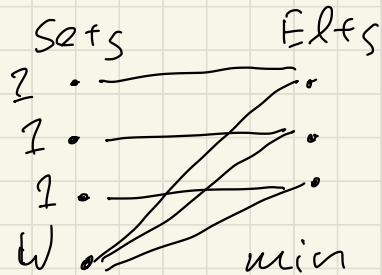
walk $\longleftrightarrow$ set covers everything



cost w

$$x_w = \begin{cases} 1 & \text{if we walk} \\ 0 & \end{cases}$$

$$x_i = \begin{cases} 1 & \text{if we wait at timestep } i. \\ 0 & \end{cases}$$



$$\min \quad W \cdot x_w + \sum_i x_i$$

s.t.

$$x_w + x_i \geq 1 \quad \forall i \in A$$

↑ train arrival time.

$$\max \quad \sum_{i \in A} y_i$$

$$\text{s.t.} \quad \sum_{i \in A} y_i \leq W$$

(x_w)

$$y_i \leq 1 \quad \forall i \in A \quad (x_i)$$

Online Fractional Algorithm

$$\begin{aligned} \min \quad & w \cdot x_w + \sum_i x_i \\ \text{s.t.} \quad & x_w + x_i \geq 1 \quad \forall i \in A \end{aligned}$$

↑ train arrival time.

Initialize $x_i \leftarrow 0$, $x_w \leftarrow 0$

For each i :

$$x_w \leftarrow \left(1 + \frac{1}{w}\right) x_w + \alpha$$

$$x_i \leftarrow 1 - x_w$$

$$y_i \leftarrow 1$$

$$\Delta \text{Primal} = \underbrace{\left(\frac{x_w}{w} + \alpha\right)w}_{\Delta x_w} + \underbrace{(1 - x_w)}_{\Delta x_i} = 1 + \alpha \cdot w$$

Goal Set α s.t. $\sum_i y_i \leq w$

Set α s.t. $x_w = 1$ after w time steps.

$$\text{After } w \text{ timesteps, } x_w = \frac{(1 + \frac{1}{w})^w - 1}{1/w\alpha} = 1$$

$$\text{Solving for } \alpha \Rightarrow 1 + \alpha w = 1 + \frac{1}{(1 + \frac{1}{w})^w - 1}$$

$$\approx \frac{e}{e-1}$$

$\Rightarrow \frac{e}{e-1}$ -comp- fractional

Rounding: Pick $\theta \in [0, 1]$ uniformly.

Walk when $x_w \geq \theta$.

wait when $x_w < \theta$.

Recap

Part 1: Greed is Good

Greedy

- Matroids, Independence Systems
- Coverage and Submodular Maximization
- Set Cover, Knapsack
- Marginal Greedy and Density Greedy
- Matching and Vertex Cover
 - First glimpse of duality

Greedy-ish: Augmenting Path Algorithm for Matching

Part 2: Stop Worrying and Relax Continuously

- Many problems can be modeled as Integer Linear Programs
- Can be solved efficiently by relaxing integrality requirements
 - Separation oracle
- LP relaxation is valid if every solution to combinatorial optimization problem has a corresponding feasible LP solution
- Optimal LP solution:
 - upper bounds OPT for maximization problems
 - lower bounds OPT for minimization problems

Rounding LPs

- Knapsack, vertex cover, set cover, min (s,t)-cut
- Threshold-based rounding
 - Round up all variables at least some threshold
 - Pick a random threshold, round up all variables at least threshold
 - Expected probability of rounding up a variable is at most some α times fractional value of variable
 - Ball with random radius (Min (s,t)-cut)

Duality

- Weak duality
- Dual-fitting
 - Vertex cover, set cover, online fractional set cover
- Primal-dual
 - Complementary slackness
 - Vertex cover, set cover
 - Cut-type constraints for connectivity problems: shortest path, MST
 - Dual growth/ascent for shortest path corresponds to growing a ball = BFS

Notes on LPs

- The same problem can admit several linear programs
- Some are better than others
 - Integrality gap (e.g. cut vs partition LPs for MST)
 - Efficient separation oracle
- Primal-dual does not need to solve LP
- Rounding does not need dual
 - Some duals are complicated (e.g. dual of triangle inequality constraints)

Theme 1: Charging

- To prove optimality/approximation ratio, need some way of charging to optimal solution
- All the different analyses is based on different characterizations of/bounds on optimal solution
- Greedy for matroids: exchange argument
- Greedy for submodular maximization: diminishing marginals
- Rounding: charge to a fractional relaxation
- Dual-fitting: charge to dual (which charges to fractional relaxation)
- Primal-dual: charging to dual via complementary slackness
- Tram Waiting

Theme 2: Don't Worry and Continuously Relax

- Fractional relaxations are very powerful!
- LP-based methods subsume many greedy and dynamic programming algorithms
- We only covered the tip of the iceberg
 - Matroid constraints can be formulated as linear constraints
 - Optimal solutions of the “matroid” polytope are integral
- Convex programming
 - Convex relaxations of submodular functions
 - Semidefinite programming

Theme 3: Limitations of Algorithms

- Greedy:  Matroid,  Independence systems
- Inapproximability (e.g. set cover, coverage maximization)
- Integrality gap for LPs
- Adversarial arguments for online algorithms

Theme 4: Problem Structure Matters

- Set Cover: $O(\log n)$ -approximation, NP-hard to do better
- Vertex Cover: 2-approximation, NP-hard
- Vertex Cover on bipartite graphs: exact algorithm, $O(n^2)$ time
- Interval Hitting Set: exact algorithm, $O(n \log n)$ time

General Skills

- How to reason about optimization problems
 - Properties of objective function (e.g. additive, submodular)
 - Properties of constraints (e.g. packing/covering, matroid)
 - Writing LP and dual helps clarify problem structure
- How to design and reason about algorithms
 - Solution quality (approximation ratio)
 - Can we do better? Lower bounds?
 - Limitations (NP-hardness, integrality gaps for LPs)

Most Importantly

- How to think for yourself and avoid being gaslit by AI
- How to approach challenging concepts and problems
- Persistence and perseverance through difficulty

Exam

- Sample and past exams will be released on Ed
- Order of questions may not necessarily indicate difficulty
- Answer box length is not an indication of length of correct answer.
 - In particular, we expect you to need extra space in case you need to modify your answer.
 - Sometimes, it's just to pad the remaining space on the page.
- Number of marks does not scale linearly with difficulty.
 - The most difficult question is meant to differentiate between 80-90 and 90-100, and has a small number of marks but can take > 30 mins
- Don't leave any question blank! We can't give you marks for a blank answer. Try to write something that demonstrates some level of understanding.

Exam Questions

- Some MCQ and short response similar to MST
- Some algorithm design and analysis questions
- Find bug in proof
- Examples, counter-examples: e.g. integrality gap
- Twists on problems/algorithms/proofs in lectures/tutorials/assignments
 - What if we drop this assumption in the theorem/lemma?
 - What if we change the algorithm slightly?
 - Special cases/generalizations of problems
- Goal:
 - Assess your ability to apply what you've learnt to new contexts
 - Some creative problem solving

How to Revise

- I attempted lots of problems and learnt *on the job*
 - I only understood linear algebra through linear programming
- That's still the best way
- Coming up with good problems is hard, however you have resources
- Try all tutorial problems
- Problems from other sources (e.g. Design of Approximation Algorithms, Approximation Algorithms books)
- Discuss with classmates
 - Ed
 - Ask the teaching staff about these problems, and your solution attempts

End of Semester Survey

Please help us improve the subject by filling out the ESS

What was good? What can be improved?

- Tutor and Tutorials
- Lectures
- Lecture notes
- Assignments

Changes From Last Year

I have continually improved my teaching from feedback

- Completely redesigned slides, materials
- BRAND NEW: my own detailed lecture notes
- More illustrations
- More examples of algorithms/problems for each paradigm
- Customizing problems to be more relatable (e.g. Tram Waiting Problem)
- Reflection on assignments
- How to approach problem solving
- Brainstorming time during lecture

<https://www.unimelb.edu.au/sls>



Ad for PhD/MCS/MIT

- Junhao Gan, Hanzhi Wang and I are seeking students to work on projects.
- Please check out our websites or drop us an email to discuss potential projects:
- Hanzhi <https://wanghzccls.github.io>
- Junhao <https://sites.google.com/site/junhogan/>
- William <https://williamumboh.com>



Thank You!

- It's been great fun teaching you
- I've (re)-learnt a lot myself
 - Finally understand proof of strong duality
 - Progress on some research problems

